

Workshop 1: GitHub & NumPy

TECH2: Introduction to Programming, Data, and Information Technology

Richard Foltyn
NHH Norwegian School of Economics

October 2, 2026

See GitHub repository for notebooks and data:

<https://github.com/richardfoltyn/TECH2-H26>

Exercise 1: Summing lists and arrays

Python's built-in function `sum()` and NumPy's function `np.sum()` can both be used to sum sequences of numbers. In this exercise, we compare the execution time of `sum()` and `np.sum()` for sequences of different lengths.

1. Create a list `lst` and a NumPy array `arr`, each of them containing the sequence of ten values 0, 1, 2, ..., 9.

Hint: You can use the list constructor `list()` and combine it with the `range()` function, which returns an object representing a range of integers.

Hint: You should create the NumPy array using `np.arange()`.

2. We want to compute the sum of integers contained in `lst` and `arr`. Use the built-in function `sum()` to sum elements of a list. For the NumPy array, use the NumPy function `np.sum()`.
3. You are interested in benchmarking which summing function is faster. Repeat the steps from above, but use the cell magic `%timeit` to time the execution of a statement as follows:

`%timeit statement`

4. Recreate the list and array to contain 100 integers starting from 0, and rerun the benchmark.
5. Recreate the list and array to contain 10,000 integers starting from 0, and rerun the benchmark.

What do you conclude about the relative performance of built-in lists vs. NumPy arrays?

Exercise 2: Maximizing quadratic utility

In economics, we usually represent an individual's preferences with a utility function, $u(\bullet)$. Assume that utility from consuming c units is given by the following quadratic function:

$$u(c) = -A(c - B)^2 + C$$

where $A > 0$, $B > 0$, and C are parameters, and c is the consumption level. The following code plots the function for specific parameter values. We will cover plotting in detail in later weeks, so you can ignore the code for now:

```
[13]: import matplotlib.pyplot as plt

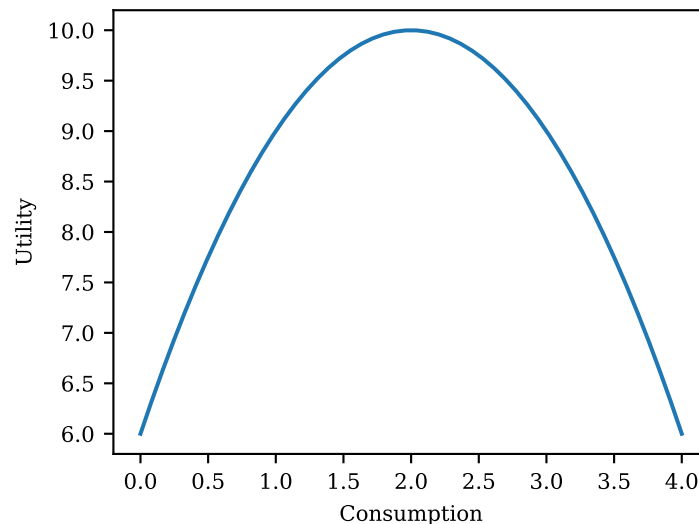
c = np.linspace(0, 4, 50)
```

```

u = -1 * (c - 2) ** 2 + 10
plt.plot(c, u)
plt.xlabel('Consumption')
plt.ylabel('Utility')

```

```
[13]: Text(0, 0.5, 'Utility')
```



In this exercise, you are asked to locate the consumption level that yields the maximum utility.

1. Define a function `util()` that takes the consumption level `c` and three additional arguments, `A`, `B`, and `C`. These are the parameters of $u(\bullet)$ defined above. The function should return the utility associated with the consumption level `c`.

Hint: Use the `**` operator for exponentiation in Python. For example, `x ** 2` computes the square of `x`.

2. Write a function `find_max_cons()` that takes a sequence of candidate consumption levels and the three parameters `A`, `B`, and `C`. The function should return the maximum utility and the corresponding consumption level. The function definition should look like this:

```

def find_max_cons(candidates, A, B, C):
    """
    Find the consumption level that maximizes utility from a
    sequence of candidates.

    Parameters
    -----
    candidates : list or array-like
        Sequence of candidate consumption levels to evaluate.
    A, B, C : float
        Parameters of the utility function.

    Returns
    -----
    u_max
        Maximized utility
    cons_max
        Consumption at which utility is maximized
    """

```

Your algorithm should perform the following steps:

1. Initialize `u_max` to `-np.inf` (negative infinity).

2. Loop through all candidate consumption levels and compute the associated utility. If this utility is larger than the current maximum value `u_max`, update `u_max` and store the associated consumption level `cons_max`.
 3. Return `u_max` and `cons_max` after the loop terminates.
3. Find the maximum:
1. Create an array `cons` containing 51 uniformly spaced candidate consumption levels on the interval $[0, 4]$.
Hint: Use `np.linspace()` for this task.
 2. Use the parameters $A = 1$, $B = 2$, and $C = 10$.
 3. Use `find_max_cons()` to compute the maximum utility and the corresponding optimal consumption level, and print the results.
4. Repeat the exercise using vectorized NumPy operations:
1. Compute and store the utility levels for *all* elements in `cons` at once by applying the formula to the entire array.
 2. Locate the index of the maximum utility level using `np.argmax()`.
 3. Use the index returned by `np.argmax()` to retrieve the maximum utility and the corresponding consumption level, and print the results.

Exercise 3: Progressive income taxation

Consider the following simplified, illustrative progressive income-tax system:

- Income up to NOK 300,000 is not taxed.
- Income from NOK 300,000 to NOK 700,000 is taxed at 20%.
- Income above NOK 700,000 is taxed at 35%.

This schedule is designed for the exercise and does not represent the actual Norwegian tax system.

1. Define a function `tax()` that takes annual income in NOK as its argument and returns the amount of tax owed. Use `if`, `elif`, and `else` to handle the three income ranges.
2. Calculate taxes for a sequence of incomes using a loop:
 1. Create an array `incomes` containing 13 evenly spaced values from NOK 0 to NOK 1,200,000 using `np.linspace()`.
 2. Create an empty array `taxes_loop` with the same shape as `incomes`.
 3. Loop over `incomes`, evaluate `tax()` for each element, and store the result in `taxes_loop`.
 4. Compute the corresponding after-tax incomes.
 5. Print a table with one row per income and columns for gross income, tax, and after-tax income. Report all amounts in NOK.
3. Define a function `tax_numpy()` that takes an array of annual incomes and returns an array containing the corresponding taxes. The function should use vectorized NumPy operations instead of a loop.

Hint: Use `np.maximum()` to prevent taxable amounts from becoming negative and `np.minimum()` to cap the income taxed at 20% at NOK 400,000.

Call `tax_numpy()` with `incomes` and compute the corresponding after-tax incomes. Use `np.allclose()` to verify that the loop and vectorized calculations produce the same taxes and after-tax incomes.